

Python Programming Guide

Small building blocks for readable, dependable programs

Values and useful structures

Python programs work with values such as strings, numbers, and booleans. Variables give those values names so that a program can describe an idea clearly. Prefer names that explain a purpose: `document_count` communicates more than a single letter. Readable code reduces the effort needed to understand what happens and why it happens.

Lists preserve an ordered collection of values. Dictionaries associate keys with values and are useful for structured information such as a document's title and page count. Sets represent unique items and make membership checks convenient. Choosing the right structure often simplifies a program more than adding clever control flow.

Functions and responsibilities

A function collects a useful operation behind a clear name. A good function usually has one responsibility, receives the values it needs, and returns a result that callers can use. Separating calculation from file access makes both operations easier to test. A text analysis function, for example, can accept a string without knowing where that string came from.

Default arguments and keyword arguments make interfaces easier to read, but mutable default values need care. Use `None` when a new list or dictionary should be created for each call. Type hints document expected inputs and outputs; they improve editor feedback and help explain an interface, while runtime validation still checks untrusted values.

Files, errors and resources

Use a context manager when working with a file so the resource closes reliably. Explicit text encodings prevent surprises when a project moves between machines. Paths should be assembled with a path library rather than by joining arbitrary strings. User-provided filenames should never be trusted as storage paths.

Errors are part of normal software behavior. Catch the errors an operation can reasonably recover from, explain the problem clearly, and preserve useful diagnostic information for developers. Avoid hiding every exception behind a successful result. If writing a file and a database record forms one operation, plan what happens when either step fails.

Testing and project habits

A useful test checks observable behavior with representative inputs. Include empty input, unusual characters, invalid values, and access boundaries where they matter. Tests should make failures easy to understand and should run without relying on a live external service. Temporary directories and isolated databases keep tests predictable.

Create a virtual environment for each project and record its dependencies. Keep secrets outside source control and document the commands needed to run the application. Consistent formatting, short modules, and meaningful names make a project easier to extend. The best Python code leaves the next reader with fewer questions.